FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
Ingeniería en Ciberseguridad

PROGRAMACIÓN III (ISWZ2102)

INFORME TÉCNICO

AVANCE DEL PROYECTO, PROGRESO 1

Análisis, diseño y selección de la solución

TEMA: SECR, Sistema de Evaluación de Conducción Responsable

Priorización del riesgo de conductores mediante estructuras de datos y algoritmos en Java

ESTUDIANTES (GRUPO 4):

Mateo Vasquez
Alex Palma
Martín Jurado
Amelio Macas
Israel Zurita

Período académico 2027-1
QUITO, ECUADOR
Octubre de 2026

Contenido

Resumen ejecutivo....2
1. Formulación del problema....3
1.1 Contexto y necesidad....3
1.2 Actores y grupos involucrados....3
1.3 Causas y consecuencias....4
1.4 Datos y procesos principales....4
1.5 Variables relevantes y restricciones....4
1.6 Relación con factores globales, sociales, culturales y económicos....5
1.7 Enunciado del problema....5
2. Alcance y módulos funcionales....6
2.1 Alcance....6
2.2 Módulo M1: Gestión de conductores y licencias....6
2.3 Módulo M2: Registro y análisis de eventos e infracciones....6
2.4 Módulo M3: Puntaje y priorización de riesgo....7
2.5 Módulo M4: Recuperación de puntos (cursos, inscripciones y pruebas)....7
2.6 Módulo M5: Alertas y reportes....8
2.7 Matriz de responsabilidades....8
3. Modelo orientado a objetos y diagrama UML preliminar....9
3.1 Herramienta y criterios....9
3.2 Clases de dominio y relaciones....9
3.3 Cambios respecto al semestre anterior....9
3.4 Correspondencia entre módulos y clases....10
4. Alternativas basadas en estructuras de datos y algoritmos....13
4.1 Detalle de complejidad por operación....14
5. Selección y fundamentación de la mejor alternativa....15
5.1 Análisis por criterios....15
5.2 Evidencia experimental....15
5.3 Decisión....16
6. Plan preliminar de implementación....17
6.1 Módulos y componentes asignados....17
6.2 Estructura inicial del repositorio GitHub....17
6.3 Cronograma hacia el Progreso 2....18
6.4 Riesgos técnicos y acciones de mitigación....18
Declaración de uso de herramientas de inteligencia artificial....19
Referencias....20
Anexo A. Reproducción de los resultados....20

Resumen ejecutivo

El presente informe documenta el Progreso 1 del proyecto de Programación III. El equipo retoma **SECR (Sistema de Evaluación de Conducción Responsable)**, desarrollado el semestre anterior como una aplicación Java con interfaz Swing orientada principalmente al registro (CRUD) de conductores, infracciones, cursos y licencias, y lo replantea como un problema de **estructuras de datos y algoritmos**: mantener actualizado el ranking de conductores de mayor riesgo, consultar el historial de infracciones por rangos de fechas, buscar conductores por prefijo de nombre, priorizar licencias próximas a caducar y asignar cupos en cursos de recuperación de puntos.

Se definen cinco módulos funcionales con sus operaciones, reglas y responsables; un modelo orientado a objetos con dos diagramas UML elaborados en PlantUML; y dos alternativas técnicamente diferenciadas. La **Alternativa A** usa arreglos dinámicos ordenados con búsqueda binaria y reordenamiento por mezcla. La **Alternativa B** combina tabla hash, árbol de prefijos (Trie), montículo binario indexado y árbol rojo-negro. Ambas se prototiparon en Java, se verificó con 16 pruebas automáticas que devuelven los mismos resultados y se midieron con un benchmark de hasta 100 000 registros.

Se selecciona la **Alternativa B**. Con 100 000 conductores, la operación crítica (actualizar un puntaje y consultar el top-10 de riesgo, 500 veces) tarda 2,3 ms frente a 26 139 ms de la Alternativa A, a cambio de aproximadamente un 23 % más de memoria en los índices. El informe cierra con el plan de implementación para el Progreso 2, la estructura del repositorio, los riesgos técnicos y la declaración de uso de inteligencia artificial.

1. Formulación del problema

1.1 Contexto y necesidad

Los siniestros de tránsito son un problema de salud pública a escala mundial. La Organización Mundial de la Salud estima que provocan alrededor de **1,19 millones de muertes al año**, que son la principal causa de muerte de niños y jóvenes de 5 a 29 años y que nueve de cada diez de esas muertes ocurren en países de ingresos bajos y medianos (World Health Organization [WHO], 2023). Ecuador refleja esta realidad: según datos de la Agencia Nacional de Tránsito (ANT), en 2024 se registraron **21 220 siniestros y 2 302 personas fallecidas**, lo que equivale aproximadamente a una muerte cada 3,8 horas; las causas más frecuentes fueron la conducción desatenta, el exceso de velocidad y el irrespeto a las señales de tránsito (El Universo, 2025).

Para desincentivar las conductas de riesgo, el país aplica un **sistema de licencias por puntos**: el conductor parte de un saldo de 30 puntos, cada contravención descuenta puntos y, al agotarlos, la licencia se suspende; los puntos pueden recuperarse mediante cursos autorizados (Asamblea Nacional del Ecuador, 2008). La evidencia internacional indica que estos sistemas reducen los siniestros, fallecidos y lesionados entre un 15 % y un 20 % al inicio, pero que el efecto se diluye en menos de dieciocho meses si no se acompaña de control y seguimiento efectivos (Castillo-Manzano & Castro-Nuño, 2012); el análisis econométrico del caso español confirma la necesidad de sostener la vigilancia (Castillo-Manzano et al., 2010).

Ese seguimiento exige información actualizada que se transforme en acciones oportunas. El semestre anterior el equipo construyó SECR como un sistema de registro; al analizarlo con criterios de Programación III se concluyó que las operaciones de mayor valor son las de **procesamiento**: saber en todo momento quiénes son los conductores de mayor riesgo, consultar el historial por periodos, priorizar las licencias por caducar y asignar cupos en los cursos. La versión anterior las resolvía recorriendo o reordenando colecciones completas, algo aceptable con decenas de registros, pero no con decenas de miles.

1.2 Actores y grupos involucrados

Tabla 1. *Actores del problema y su interés en la solución*

Actor	Rol en el problema	Interés principal
Agente u operador de tránsito	Registra infracciones y consulta el historial del conductor.	Registro rápido y validado; consulta inmediata por licencia.
Analista o administrador de la autoridad de tránsito	Configura catálogos, revisa rankings y genera reportes.	Identificar conductores de alto riesgo y licencias por caducar para intervenir a tiempo.
Conductor	Sujeto evaluado; acumula infracciones, rinde pruebas y renueva su licencia.	Conocer su puntaje, recuperar puntos y renovar sin demoras.
Centro de capacitación o instructor	Dicta los cursos de recuperación y aplica las pruebas.	Asignación ordenada de cupos y evaluación objetiva.
Sociedad (peatones, ciclistas, otros conductores)	Beneficiario indirecto de vías más seguras.	Reducción de siniestros por conductas reincidentes.

1.3 Causas y consecuencias

Causas. (1) Información de infracciones, cursos y renovaciones dispersa; (2) puntaje recalculado manualmente o por lotes; (3) ausencia de un mecanismo que muestre de inmediato a los conductores de mayor riesgo; (4) consultas resueltas con recorridos lineales y reordenamientos completos; y (5) control reactivo de la caducidad de licencias.

Consecuencias. La intervención sobre conductores reincidentes llega tarde, lo que debilita el efecto disuasorio que el sistema de puntos busca (Castillo-Manzano & Castro-Nuño, 2012); circulan conductores con licencias vencidas; los cupos de los cursos se asignan sin un criterio verificable; y los reportes se vuelven lentos a medida que crece la base de datos.

1.4 Datos y procesos principales

Datos: conductor (cédula, nombre, fecha de nacimiento, estado, puntaje); licencia (número, categoría B, C o E, vigencia de 5 años); tipo de evento (puntos base); evento (fecha, ubicación, gravedad de 1 a 5); curso (puntos recuperables, porcentaje mínimo, cupo); pregunta, prueba, renovación y alerta.

Proceso central: registro del conductor → registro de infracción → cálculo de la deducción → recálculo del puntaje y reclasificación del riesgo → generación de alertas → inscripción en un curso → prueba de recuperación → acreditación de puntos → renovación de la licencia cuando corresponde.

1.5 Variables relevantes y restricciones

Tabla 2. *Variables del problema*

Variable	Dominio / fórmula	Uso
Puntaje de conducción	Real en [0, 100]. Puntaje = 100 − Σ deducciones + recuperación + bonificación	Clasificación de riesgo y prioridad
Deducción por evento	puntosBase(tipo) × gravedad, gravedad ∈ {1,…,5}	Actualización del puntaje
Nivel de riesgo	Verde ≥ 80; amarilla ≥ 60; naranja ≥ 40; roja < 40	Alertas y semáforo
Días para vencer	fechaVencimiento − hoy (aviso con 30 días)	Priorización de caducidad
Bonificación	5 puntos por año sin infracciones	Incentivo al buen comportamiento
Aprobación de prueba	Nota ≥ 70 % (configurable por curso)	Recuperación de 20 o 30 puntos
Volumen	n conductores (10^3 a 10^5), m eventos, k tamaño del ranking	Análisis de complejidad y escalabilidad

Escala del puntaje. SECR usa un índice normalizado de 0 a 100 en lugar de los 30 puntos legales para ganar granularidad en la clasificación del riesgo; la conversión es lineal (100 puntos SECR equivalen a 30 puntos legales).

Restricciones del dominio: el conductor debe tener al menos 18 años y una cédula válida (algoritmo de dígito verificador módulo 10); el número de licencia es único con formato B-001 a E-999999; la licencia solo se renueva si está vencida, el conductor está activo y tiene al menos 50 puntos; un evento no puede tener fecha futura ni anterior a cinco años; al llegar a 0 puntos el conductor queda inactivo; no se elimina un conductor con historial.

Restricciones del proyecto: Java 17 o superior sin librerías externas; reutilización del código anterior; datos en memoria en esta fase; equipo de cinco integrantes; escalar a 10^5 conductores en un computador personal.

1.6 Relación con factores globales, sociales, culturales y económicos

Global y social: la reducción de muertes por siniestros de tránsito es una meta de la Agenda 2030 (ODS 3.6) y del Decenio de Acción para la Seguridad Vial 2021-2030, que plantea reducirlas a la mitad para 2030 (WHO, 2023). **Cultural:** varias de las causas principales, como la conducción desatenta o el uso del celular, son conductas normalizadas; SECR combina la sanción con incentivos (bonificación por buen comportamiento y cursos de recuperación) para reforzar una cultura de conducción responsable. **Económico:** los siniestros generan costos de atención médica y pérdida de productividad para las familias y el Estado (WHO, 2023); priorizar la intervención sobre los conductores de mayor riesgo optimiza el uso de recursos de control. No se identifica una relación ambiental directa y justificable, por lo que no se considera.

1.7 Enunciado del problema

¿Cómo organizar en memoria la información de conductores, infracciones y cursos para que las operaciones críticas del sistema de puntos (búsqueda, priorización por riesgo, consulta por periodos y asignación de cupos) respondan en tiempo constante o logarítmico a medida que crece el número de registros, manteniendo la coherencia con las reglas de negocio?

2. Alcance y módulos funcionales

2.1 Alcance

Incluye: gestión de conductores y licencias, registro y análisis de infracciones, cálculo y priorización del riesgo, cursos de recuperación con cola de inscripción y pruebas, alertas y reportes; interfaz gráfica Swing y consola; estructuras de datos propias (Trie, montículo indexado) y de la Java Collections Framework justificadas por su complejidad.

No incluye en esta fase: persistencia en base de datos (se evaluará archivo CSV o serialización en el Progreso 2), integración con sistemas reales de la ANT, autenticación de usuarios ni despliegue web.

En cada módulo se marcan con **[ALG]** las operaciones algorítmicas propias del dominio (búsqueda, ordenamiento, priorización, filtrado o asignación), que van más allá del CRUD.

2.2 Módulo M1: Gestión de conductores y licencias

Propósito y actor	Mantener el padrón de conductores y sus licencias y permitir localizarlos de forma inmediata. Actor principal: operador de tránsito.
Datos	Conductor, Licencia, RenovacionLicencia. Índices: tabla hash por licencia y Trie por palabras del nombre.
Operaciones	• Registrar conductor con validación de cédula (módulo 10), edad y formato de licencia, e indexarlo. • [ALG] Buscar por número de licencia en O(1) promedio y por cédula. • [ALG] Búsqueda incremental por prefijo de nombre o apellido (autocompletado con Trie). • [ALG] Filtrar conductores por categoría, estado y licencias vencidas. • Renovar licencia (nueva vigencia de 5 años) y registrar el historial de renovaciones. • Actualizar datos, cambiar estado y eliminar (solo sin historial).
Reglas de negocio	Mayor de 18 años; cédula válida; licencia y cédula únicas; la renovación exige licencia vencida, conductor activo y puntaje ≥ 50; no se elimina un conductor con infracciones, pruebas o renovaciones.
Relación con otros módulos	Provee conductores a M2, M3, M4 y M5. Recibe de M3 el cambio de estado cuando el puntaje llega a 0.
Responsable	**Mateo Vasquez**

2.3 Módulo M2: Registro y análisis de eventos e infracciones

Propósito y actor	Registrar las infracciones, calcular su impacto en puntos y analizar el historial. Actor principal: agente de tránsito.
Datos	EventoConduccion, Infraccion, TipoEvento. Índice temporal: árbol rojo-negro (TreeMap) por fecha.
Operaciones	• Registrar infracción y calcular la deducción (puntos base × gravedad). • [ALG] Consultar eventos por rango de fechas, global o por conductor, en O(log d + k). • [ALG] Ordenar eventos por gravedad y fecha con ordenamiento por mezcla

	estable. • [ALG] Identificar la infracción más frecuente (conteo con tabla hash) y el top-k de ubicaciones críticas. • Administrar el catálogo de tipos de evento (crear, editar, activar o inactivar).
Reglas de negocio	Solo conductores activos y con puntos; fecha no futura ni anterior a 5 años; gravedad de 1 a 5; solo tipos activos; un tipo ya usado no se elimina, se inactiva.
Relación con otros módulos	Usa M1 para obtener el conductor; notifica cada deducción a M3; alimenta las estadísticas de M5.
Responsable	**Alex Palma**

2.4 Módulo M3: Puntaje y priorización de riesgo

Propósito y actor	Mantener actualizado el puntaje y el orden de riesgo de todos los conductores. Actor principal: analista de la autoridad de tránsito.
Datos	HistorialConduccion, puntaje actual, nivel de alerta. Estructura: montículo binario mínimo indexado (licencia → posición).
Operaciones	• Recalcular el puntaje: 100 − deducciones + recuperación + bonificación, acotado a [0, 100]. • [ALG] Actualizar la prioridad del conductor en el montículo en O(log n) tras cada evento o curso. • [ALG] Obtener el top-k de conductores de mayor riesgo en O(k log k), sin reordenar la colección. • [ALG] Clasificar el riesgo en semáforo (verde, amarilla, naranja, roja). • Aplicar la bonificación por buen comportamiento (5 puntos por año sin infracciones).
Reglas de negocio	Puntaje en [0, 100]; con 0 puntos el conductor pasa a inactivo; mayor riesgo = menor puntaje; en empate se ordena por número de licencia para que el resultado sea determinista.
Relación con otros módulos	Recibe actualizaciones de M2 (deducciones) y M4 (recuperaciones); consulta M1; entrega el ranking a M5.
Responsable	**Martín Jurado**

2.5 Módulo M4: Recuperación de puntos (cursos, inscripciones y pruebas)

Propósito y actor	Gestionar los cursos de recuperación, asignar cupos y evaluar a los conductores. Actores: conductor e instructor.
Datos	Curso, Pregunta, Prueba, EvaluacionPrueba e InscripcionCurso (nueva). Estructura: cola FIFO (ArrayDeque) por curso.
Operaciones	• [ALG] Inscribir al conductor en la cola FIFO del curso. • [ALG] Asignar cupos por orden de llegada (desencolar hasta el cupo máximo). • [ALG] Generar una prueba aleatoria con el algoritmo de Fisher-Yates (Durstenfeld, 1964). • Calificar la prueba y acreditar los puntos si la nota alcanza el mínimo. • [ALG] Reporte del curso con mayor porcentaje de aprobación.

	• Administrar cursos y banco de preguntas.
Reglas de negocio	Solo conductores con puntaje menor a 100; curso activo; aprobación ≥ 70 %; recupera 20 (básico) o 30 (avanzado) puntos sin superar 100; un conductor no puede estar dos veces en la misma cola; no se elimina un curso con preguntas.
Relación con otros módulos	Consulta M1; notifica las recuperaciones a M3; entrega estadísticas a M5.
Responsable	**Amelio Macas**

2.6 Módulo M5: Alertas y reportes

Propósito y actor	Convertir los datos en información para decidir: alertas, reportes individuales y estadísticas. Actor principal: administrador o analista.
Datos	Alerta, Licencia y resultados de M1 a M4. Estructura: cola de prioridad (min-heap) por fecha de vencimiento.
Operaciones	• [ALG] Alertas de caducidad priorizadas por días para vencer (las más urgentes primero). • Reporte individual con historial de eventos, pruebas, renovaciones y bonificación. • Reporte estadístico general (activos e inactivos, promedios, totales). • [ALG] Ranking ordenable por varios criterios (puntaje, luego nombre) con ordenamiento estable. • Exportar reportes a CSV (Progreso 2).
Reglas de negocio	Aviso de caducidad con 30 días de anticipación; alerta roja con puntaje menor a 40; los reportes solo usan datos registrados.
Relación con otros módulos	Consume información de M1, M2, M3 y M4.
Responsable	**Israel Zurita**

2.7 Matriz de responsabilidades

Tabla 3. *Asignación preliminar de responsabilidades (R = responsable, A = apoyo)*

Integrante	M1	M2	M3	M4	M5	Responsabilidad transversal
Mateo Vasquez	R		A			Validaciones y enums del modelo
Alex Palma		R			A	Pruebas de las consultas por rango
Martín Jurado	A		R			Integración de las estructuras de datos
Amelio Macas				R	A	Administración del repositorio Git
Israel Zurita		A		A	R	Benchmark, pruebas e interfaz de reportes

3. Modelo orientado a objetos y diagrama UML preliminar

3.1 Herramienta y criterios

Los diagramas se elaboraron con **PlantUML** (versión 1.2022.12) y se mantienen como código fuente en `docs/uml/*.puml` dentro del repositorio, de modo que cada integrante pueda revisarlos, comentarlos y versionarlos con Git. Se partió de las clases existentes del semestre anterior (estereotipo «existente») y se añadieron las clases que exigen los nuevos módulos (estereotipo «nueva»). Se usan herencia, composición y agregación solo donde el dominio lo justifica.

3.2 Clases de dominio y relaciones

Tabla 4. *Justificación de las relaciones del modelo*

Relación	Tipo y multiplicidad	Justificación
Conductor – Licencia	Composición 1 : 1	La licencia no existe en el sistema sin su titular; se crea y elimina con él.
Conductor – HistorialConduccion	Composición 1 : 1	El historial es parte del conductor y concentra el cálculo del puntaje.
HistorialConduccion – EventoConduccion	Composición 1 : 0..*	Los eventos pertenecen al historial de un único conductor.
EventoConduccion ← Infraccion	Herencia	EventoConduccion es abstracta y define la deducción (puntosBase × gravedad); Infraccion es el tipo concreto que se registra hoy y deja abierta la extensión a otros eventos.
EventoConduccion – TipoEvento	Asociación 0..* : 1	Muchos eventos comparten un tipo del catálogo, que vive de forma independiente.
Conductor – RenovacionLicencia	Composición 1 : 0..*	Bitácora de renovaciones propia del conductor.
Conductor – Prueba – Curso	Asociación 1 : 0.. *y 0..* : 1	Un conductor rinde varias pruebas; cada prueba evalúa un curso.
Prueba – EvaluacionPrueba	Composición 1 : 0..1	Resumen de la calificación generado por la propia prueba.
Curso – Pregunta	Composición 1 : 0..*	El banco de preguntas de un curso se elimina con el curso.
InscripcionCurso – Conductor / Curso	Asociación 0..* : 1	Nueva clase: elemento de la cola FIFO de cupos del módulo M4.
Alerta – Conductor	Asociación 0..* : 1	Las alertas se generan a partir del estado de un conductor.

3.3 Cambios respecto al semestre anterior

Tabla 5. *Evolución del modelo*

Cambio	Motivo
Nuevos enums EstadoConductor, CategoriaLicencia y NivelAlerta en lugar de String	Evitar valores inválidos y comparaciones de texto dispersas.

Cambio	Motivo
Nueva clase InscripcionCurso y atributo cupoMaximo en Curso	Soportar la cola de inscripción y la asignación de cupos (M4).
Nuevo método Licencia.diasParaVencer()	Clave de prioridad para las alertas de caducidad (M5).
Nuevo GestorRiesgo con MonticuloIndexado	Separar la priorización del registro de conductores (M3).
GestorConductores con HashMap + Trie y GestorEventos con IndiceTemporalEventos	Estructuras de la alternativa seleccionada (sección 5).

3.4 Correspondencia entre módulos y clases

Tabla 6. *Trazabilidad módulo → clases*

Módulo	Clases de dominio	Clases de negocio y estructuras
M1 Conductores y licencias	Conductor, Licencia, RenovacionLicencia, CategoriaLicencia, EstadoConductor	GestorConductores, Trie
M2 Eventos e infracciones	EventoConduccion, Infraccion, TipoEvento	GestorEventos, IndiceTemporalEventos, OrdenamientoMezcla
M3 Puntaje y riesgo	HistorialConduccion, NivelAlerta	GestorRiesgo, MonticuloIndexado
M4 Recuperación de puntos	Curso, Pregunta, Prueba, EvaluacionPrueba, InscripcionCurso	GestorCursos (ArrayDeque)
M5 Alertas y reportes	Alerta, Licencia	GestorReportes (PriorityQueue), OrdenamientoMezcla

Las figuras 1 y 2 se presentan en páginas horizontales para mejorar su lectura; las versiones vectoriales (SVG) están en `docs/uml/`.

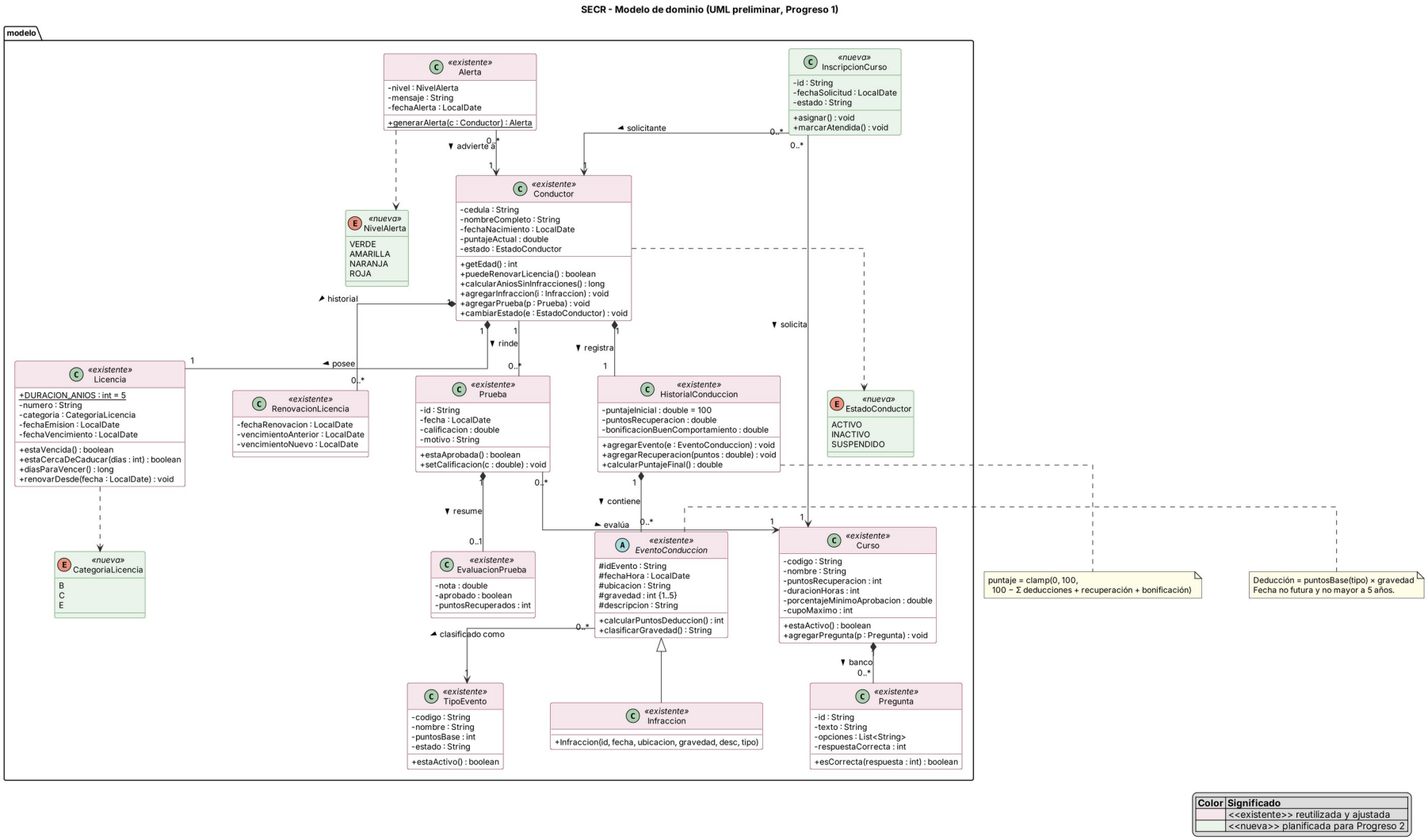


Figura 1. *Diagrama de clases preliminar: modelo de dominio de SECR (PlantUML).*

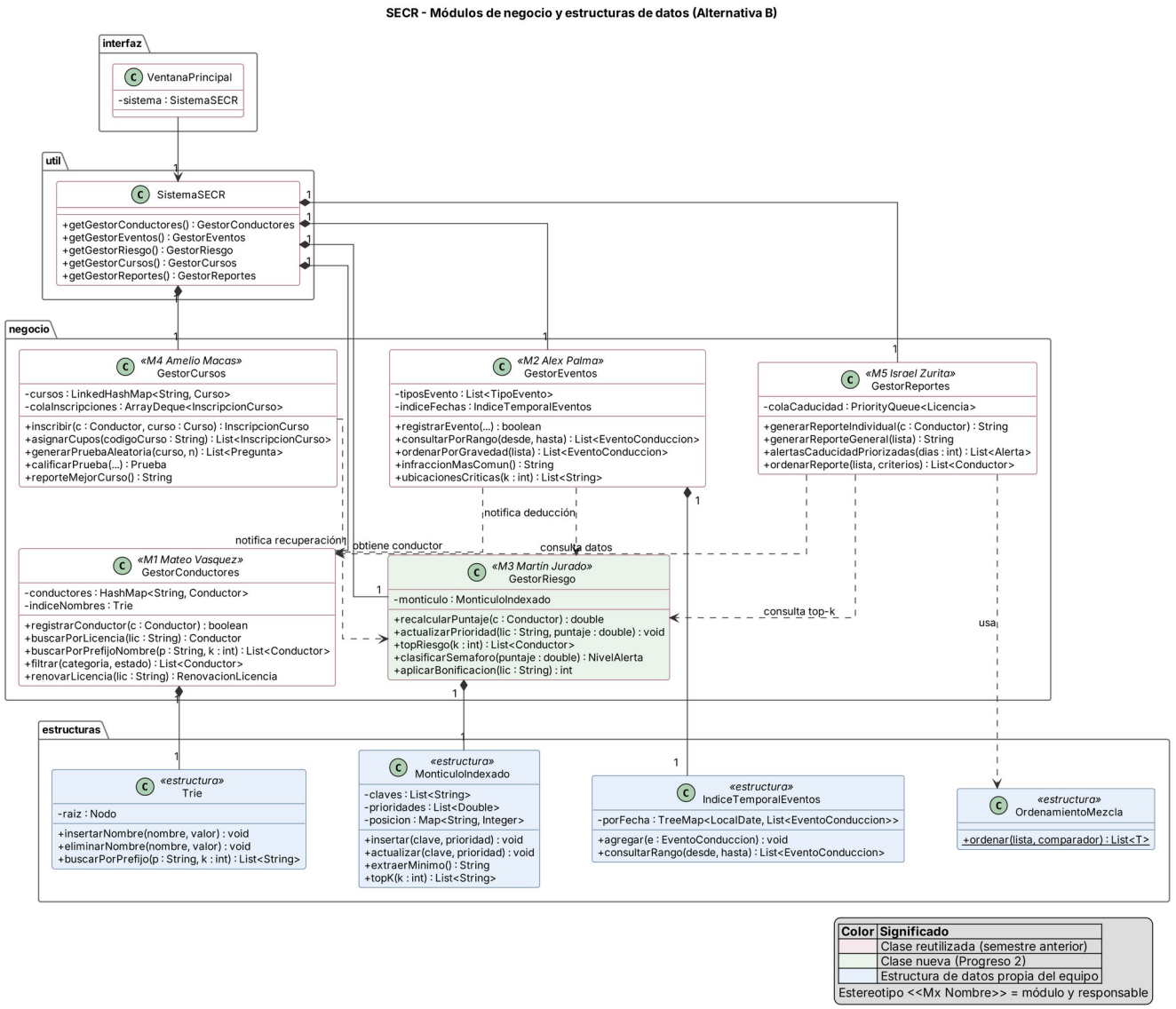


Figura 2. *Diagrama de clases: módulos de negocio, responsables y estructuras de datos de la alternativa seleccionada.*

4. Alternativas basadas en estructuras de datos y algoritmos

Las alternativas resuelven el componente más relevante de la solución: **el núcleo de consulta y priorización de SECR**, formado por cuatro operaciones que usan todos los módulos: (1) búsqueda de un conductor por licencia, (2) búsqueda por prefijo de nombre, (3) ranking top-k de riesgo con puntajes que cambian en cada infracción o curso y (4) consulta del historial de eventos por rango de fechas. Notación: n = conductores, m = eventos, d = fechas distintas, k = resultados, L y P = longitud de la clave y del prefijo, R = nodos visitados del subárbol del prefijo.

La tabla 7 resume ambas alternativas en el formato solicitado y se presenta en una página horizontal; la tabla 8 detalla la complejidad de cada operación.

Tabla 7. *Alternativas planteadas (formato de la consigna)*

Alternativa	Estructura(s)	Algoritmo(s)	Operación que resuelve	Complejidad temporal	Complejidad espacial	Ventajas / limitaciones
A Arreglos ordenados	• ArrayList ordenado por licencia • ArrayList ordenado por palabra del nombre • ArrayList de eventos ordenado por fecha	• Búsqueda binaria (límite inferior) • Inserción ordenada con desplazamiento • Ordenamiento por mezcla (TimSort) para el ranking • Barrido secuencial del rango	Búsqueda por licencia y por prefijo; ranking top-k; consulta por rango de fechas	• Registrar: O(n) • Buscar licencia: O(log n) • Prefijo: O(log n + k) • Actualizar puntaje: O(log n) • Top-k: O(n log n) por consulta • Registrar evento: O(m) • Rango: O(log m + k)	O(n + m) para los índices; O(n) adicional en cada ranking (copia a ordenar)	**Ventajas:** simple, poca memoria, buena localidad de caché; prefijo y rango eficientes. **Limitaciones:** inserciones O(n); el orden por riesgo se pierde con cada actualización y obliga a reordenar todo.
B Estructuras indexadas y jerárquicas	• Tabla hash (HashMap) licencia → conductor • Trie de palabras del nombre • Montículo binario mínimo indexado (arreglo + mapa de posiciones) • Árbol rojo-negro (TreeMap) fecha → eventos	• Hashing • Recorrido DFS del subárbol del prefijo • Flotar/hundir (sift-up/down) y top-k con cola frontera • Búsqueda en árbol balanceado + recorrido en orden	Las mismas cuatro operaciones	• Registrar: O(1) + O(L) + O(log n) • Buscar licencia: O(1) promedio • Prefijo: O(P + R) • Actualizar puntaje: O(log n) • Top-k: O(k log k) • Registrar evento: O(log d) • Rango: O(log d + k)	O(n + m) asintótico, con constantes mayores: nodos del Trie y mapa de posiciones (≈ 1,2 veces la memoria de A)	**Ventajas:** todas las operaciones críticas en O(1) u O(log n); el ranking siempre está listo. **Limitaciones:** más memoria; mayor complejidad de implementación; los tres índices deben mantenerse sincronizados.

Nota. Elaboración propia. Complejidades según Cormen et al. (2022) y Sedgewick y Wayne (2011); Trie según Fredkin (1960); montículo según Williams (1964); árbol rojo-negro según Guibas y Sedgewick (1978).

4.1 Detalle de complejidad por operación

Tabla 8. *Comparación de complejidad temporal por operación*

Operación	Alternativa A	Alternativa B	Frecuencia esperada
Registrar conductor	O(n): desplazamiento en el arreglo	O(1) promedio + O(L) + O(log n)	Media
Buscar por licencia	O(log n)	O(1) promedio	Muy alta
Buscar por prefijo de nombre	O(log n + k)	O(P + R)	Alta (autocompletado)
Actualizar puntaje (cada infracción o curso)	O(log n), pero invalida el orden por riesgo	O(log n) y el orden se mantiene	Muy alta
Top-k de mayor riesgo	O(n log n) en cada consulta	O(k log k)	Muy alta (panel del analista)
Registrar evento	O(m)	O(log d)	Muy alta
Eventos en un rango de fechas	O(log m + k)	O(log d + k)	Alta

Ambas alternativas comparten algoritmos que no dependen de la elección: validación de cédula por dígito verificador, conteo de frecuencias con tabla hash (infracción más común y ubicaciones críticas), cola FIFO para cupos, Fisher-Yates para pruebas aleatorias y cola de prioridad para la caducidad de licencias.

5. Selección y fundamentación de la mejor alternativa

5.1 Análisis por criterios

Se evaluó cada alternativa de 1 (deficiente) a 5 (excelente) en los criterios de la consigna. Los pesos reflejan que SECR es un sistema de consulta intensiva: los puntajes cambian con cada infracción o curso y el analista consulta el ranking de forma continua, por lo que la adecuación y el tiempo de respuesta pesan más que la memoria.

Tabla 9. *Matriz de decisión ponderada*

Criterio	Peso	A	B	Argumento técnico
Adecuación al problema	25 %	3	5	El problema exige que el ranking de riesgo esté siempre disponible mientras los puntajes cambian; B lo mantiene por construcción, A debe reconstruirlo.
Tiempo de ejecución / complejidad	25 %	2	5	B es O(1) u O(log n) en todas las operaciones críticas; A tiene inserciones O(n) y ranking O(n log n).
Uso de memoria	10 %	5	4	A es más compacta; B usa ≈ 23 % más por los nodos del Trie y el mapa de posiciones (28 MB frente a 23 MB con $n = 10^5$), un costo aceptable en un equipo actual.
Escalabilidad	15 %	2	5	Al multiplicar n por 100, la operación crítica de A crece ≈ 426 veces; la de B, ≈ 2,8 veces.
Mantenibilidad	15 %	4	4	A es más simple; B encapsula cada estructura detrás de un gestor y una interfaz común, con pruebas automáticas.
Dificultad razonable de implementación	10 %	5	3	B exige implementar el Trie y el montículo indexado y sincronizar tres índices; ya existe un prototipo probado.
Puntaje ponderado	**100 %**	**3,15**	**4,55**	**B supera a A en los criterios de mayor peso.**

5.2 Evidencia experimental

Para que la decisión no dependa solo del análisis asintótico, ambas alternativas se implementaron en Java detrás de las interfaces `IndiceConductores` e `IndiceEventos` (paquete `estructuras.comparacion`). Primero se verificó con 16 pruebas automáticas (`benchmark.PruebasEstructuras`) que A y B devuelven exactamente los mismos resultados; luego se midió el tiempo con `benchmark.BenchmarkAlternativas` usando datos sintéticos con semilla fija, calentamiento previo de la JVM y la mediana de cinco repeticiones (OpenJDK 21, Linux, 2 vCPU).

Tabla 10. *Resultados del benchmark con n = 100 000 (mediana, en milisegundos)*

Operación	A (ms)	B (ms)	Resultado
Registro de 100 000 conductores	3 741,0	795,5	B es 4,7× más rápida
10 000 búsquedas por licencia	16,9	4,5	B es 3,7× más rápida
2 000 búsquedas por prefijo	4,1	3,3	B es 1,2× más rápida
500 × (actualizar puntaje + top-10)	26 138,9	2,3	B es 11 320× más rápida
Registro de 100 000 eventos	323,5	15,2	B es 21,3× más rápida

Operación	A (ms)	B (ms)	Resultado
1 000 consultas por rango de 30 días	38,8	7,8	B es 5,0× más rápida
Memoria aproximada del índice (MB)	23,1	28,4	B usa 23 % más

Nota. Elaboración propia a partir de `docs/benchmark/resultados.csv`. Los valores absolutos dependen del equipo; lo relevante es cómo crecen al aumentar n.

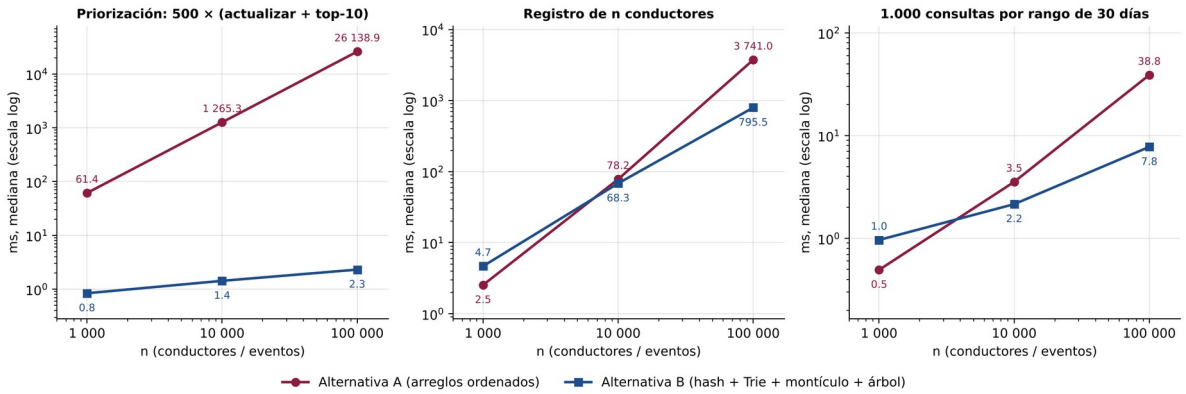


Figura 3. *Crecimiento del tiempo de A y B para n = 1 000, 10 000 y 100 000 (ejes logarítmicos).*

Lectura de los resultados. La diferencia decisiva está en la priorización: con n = 1 000 la Alternativa A ya es 73 veces más lenta y con n = 100 000 pasa a necesitar 26,1 segundos frente a 2,3 ms, porque reordena toda la colección en cada consulta. La inserción ordenada de A crece de forma cuadrática en el total (O(n) por cada registro), visible en el registro de conductores y de eventos.

También hay resultados favorables a A que se reportan con transparencia: con 1 000 registros A es más rápida al registrar conductores y en las consultas por rango (menos sobrecarga por objeto), la búsqueda por prefijo es prácticamente equivalente en ambas y A ocupa menos memoria. Estas ventajas son pequeñas en términos absolutos (décimas de milisegundo) y se invierten al crecer los datos.

Limitaciones de la medición: los datos son sintéticos, se usó un solo equipo y la JVM introduce variaciones por compilación JIT y recolección de basura; por eso se reportan medianas y se interpretan tendencias, no valores exactos.

5.3 Decisión

Se implementará la Alternativa B (tabla hash + Trie + montículo binario indexado + árbol rojo-negro), con un puntaje ponderado de 4,55 frente a 3,15. Es la única que mantiene el ranking de riesgo siempre actualizado con costo logarítmico, escala a 10^5 conductores con tiempos de milisegundos y su costo adicional de memoria (≈ 23 %) es aceptable. Para mitigar su mayor complejidad, cada estructura queda encapsulada en un gestor, con pruebas automáticas de equivalencia que se ejecutarán en cada cambio.

El ordenamiento por mezcla de la Alternativa A no se descarta: se reutilizará en M2 y M5 para ordenar listados y reportes por varios criterios, donde la estabilidad del algoritmo es una ventaja.

6. Plan preliminar de implementación

6.1 Módulos y componentes asignados

Tabla 11. *Componentes a implementar en el Progreso 2 por integrante*

Integrante	Módulo	Componentes a implementar
Mateo Vasquez	M1	Integrar HashMap + Trie en GestorConductores; búsqueda por prefijo con autocompletado en PanelConductores; filtros por categoría y estado; enums EstadoConductor y CategoriaLicencia.
Alex Palma	M2	Integrar IndiceTemporalEventos en GestorEventos; consulta por rango de fechas en PanelEventos; ordenamiento por gravedad con OrdenamientoMezcla; top-k de ubicaciones críticas.
Martín Jurado	M3	Crear GestorRiesgo con MonticuloIndexado; notificaciones desde M2 y M4; semáforo con NivelAlerta; panel de ranking top-k.
Amelio Macas	M4	Clase InscripcionCurso y cola FIFO con cupos en GestorCursos; prueba aleatoria con Fisher-Yates; flujo de calificación; administración del repositorio.
Israel Zurita	M5	Alertas de caducidad con PriorityQueue y diasParaVencer(); reportes ordenados por varios criterios; exportación CSV; mantenimiento del benchmark y las pruebas.

6.2 Estructura inicial del repositorio GitHub

El repositorio parte del proyecto del semestre anterior (*TallerEntregaFinal*) y ya contiene los prototipos de la alternativa seleccionada, el benchmark, los diagramas y el README con las instrucciones de compilación y ejecución:

```
SECR/
├── README.md                Descripción, integrantes, cómo compilar y ejecutar
├── .gitignore               Excluye out/, *.class y archivos del IDE
├── docs/
│   ├── informe/             Informe técnico del Progreso 1 (PDF)
│   ├── uml/                 Diagramas .puml y exportados .png/.svg
│   └── benchmark/           resultados.csv, grafico_benchmark.png
└── src/
    ├── modelo/              Clases de dominio (reutilizadas)
    ├── negocio/             Gestores de cada módulo
    ├── interfaz/            Interfaz gráfica Swing (reutilizada)
    ├── util/                SistemaSECR (fachada)
    ├── estructuras/         Trie, MonticuloIndexado, IndiceTemporalEventos… (nuevo)
    │   └── comparacion/     Alternativas A y B con interfaz común (nuevo)
    └── benchmark/           Pruebas automáticas y benchmark (nuevo)
```

Flujo de trabajo: rama `main` (entregas), rama `develop` (integración) y una rama `feature/` por módulo; todo cambio entra mediante *pull request* revisado por otro integrante y con las pruebas automáticas en verde.

6.3 Cronograma hacia el Progreso 2

Tabla 12. *Cronograma preliminar*

Semana	Actividad	Responsables
1	Incorporar las observaciones del docente sobre el Progreso 1; crear ramas; migrar estados a enums.	Todos
2	Integrar Trie y HashMap (M1) e IndiceTemporalEventos (M2) en los gestores.	M. Vasquez, A. Palma
3	Crear GestorRiesgo con el montículo indexado (M3) y conectar las notificaciones de M2 y M4.	M. Jurado
4	Cola de inscripción con cupos y pruebas aleatorias (M4); alertas priorizadas (M5).	A. Macas, I. Zurita
5	Integración en la interfaz Swing, pruebas de regresión y nueva ejecución del benchmark.	Todos
6	Ajustes finales, actualización de UML e informe, ensayo de la presentación.	Todos

6.4 Riesgos técnicos y acciones de mitigación

Tabla 13. *Riesgos técnicos identificados*

Riesgo	Prob.	Impacto	Acción de mitigación
Desincronización entre los índices (HashMap, Trie y montículo) al actualizar o eliminar un conductor.	Media	Alto	Centralizar toda modificación en GestorConductores y GestorRiesgo; pruebas automáticas de equivalencia y de consistencia tras cada operación.
La integración de nuevas estructuras rompe funciones de la interfaz Swing heredada.	Media	Alto	Refactorización incremental por módulo, en ramas separadas, con pruebas de regresión manuales de cada panel antes de fusionar.
Conflictos de fusión en Git al trabajar cinco personas sobre los mismos gestores.	Alta	Medio	Una rama por módulo, commits pequeños, pull requests revisados y fusión frecuente a develop.
Errores sutiles en el montículo indexado (posiciones desactualizadas, empates).	Media	Alto	Desempate determinista por licencia; prueba contra ordenamiento completo con miles de operaciones aleatorias (ya implementada).
Consumo de memoria del Trie con muchos nombres.	Baja	Medio	Normalizar claves (minúsculas, sin tildes), indexar solo palabras del nombre y medir memoria en el benchmark.
Pérdida de datos al cerrar la aplicación (datos en memoria).	Alta	Medio	Fuera del alcance del Progreso 1; evaluar exportación e importación CSV o serialización en el Progreso 2.
Mediciones de rendimiento poco confiables.	Media	Bajo	Calentamiento de la JVM, semilla fija, mediana de 5 repeticiones y análisis de tendencias, no de valores absolutos.

Declaración de uso de herramientas de inteligencia artificial

El equipo utilizó una herramienta de IA generativa como apoyo, conforme a lo permitido por la consigna. Todo contenido incorporado fue revisado, adaptado y comprendido por los integrantes, quienes asumen la responsabilidad académica y técnica sobre el código, los diagramas, las referencias y las decisiones de diseño.

Tabla 14. *Uso de herramientas de IA*

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
Claude (Anthropic)	Apoyo para estructurar el informe a partir de la consigna y del código del semestre anterior; revisión de redacción.	Borradores de las secciones 1 a 6, ajustados por el equipo.	Lectura y corrección por cada responsable de módulo; contraste de cifras y referencias con las fuentes originales (enlaces y DOI).
Claude (Anthropic)	Apoyo en el prototipo Java de las alternativas A y B (Trie, montículo indexado, índice temporal) y del benchmark.	Paquetes estructuras, estructuras.comparacion y benchmark.	Compilación y ejecución de 16 pruebas automáticas; revisión del código y de las complejidades por el equipo; repetición del benchmark.
Claude (Anthropic)	Borrador de los diagramas UML en PlantUML y de la presentación.	Diagramas de las figuras 1 y 2; diapositivas.	Revisión de clases, multiplicidades y correspondencia con los módulos; ensayo de la exposición.

Referencias

Aguas Bucheli, L. F. (2024). *Estructuras de datos para todos: Ejercicios y teoría con Java*. Our Knowledge Publishing.

Asamblea Nacional del Ecuador. (2008, 7 de agosto). *Ley Orgánica de Transporte Terrestre, Tránsito y Seguridad Vial* (Registro Oficial Suplemento 398, con reformas).

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Castillo-Manzano, J. I., & Castro-Nuño, M. (2012). Driving licenses based on points systems: Efficient road safety strategy or latest fashion in global transport policy? A worldwide meta-analysis. *Transport Policy, 21*, 191–201. https://doi.org/10.1016/j.tranpol.2012.02.003

Castillo-Manzano, J. I., Castro-Nuño, M., & Pedregal, D. J. (2010). An econometric analysis of the effects of the penalty points system driver's license in Spain. *Accident Analysis & Prevention, 42*(4), 1310–1319. https://doi.org/10.1016/j.aap.2010.02.009

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Durstenfeld, R. (1964). Algorithm 235: Random permutation. *Communications of the ACM, 7*(7), 420. https://doi.org/10.1145/364520.364540

El Universo. (2025, 22 de enero). *Durante 2024 aumentaron los siniestros de tránsito en Ecuador, pero se redujo la cantidad de fallecidos y lesionados, según la ANT*. https://www.eluniverso.com/noticias/ecuador/siniestros-de-transito-en-ecuador-durante-el-2024-nota/

Fredkin, E. (1960). Trie memory. *Communications of the ACM, 3*(9), 490–499. https://doi.org/10.1145/367390.367400

Guibas, L. J., & Sedgewick, R. (1978). A dichromatic framework for balanced trees. En *19th Annual Symposium on Foundations of Computer Science* (pp. 8–21). IEEE. https://doi.org/10.1109/SFCS.1978.3

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM, 7*(6), 347–348.

World Health Organization. (2023). *Global status report on road safety 2023*. https://www.who.int/publications/i/item/9789240086517

Anexo A. Reproducción de los resultados

Desde la raíz del repositorio, con JDK 17 o superior:

```
javac -encoding UTF-8 -d out $(find src -name "*.java")
java -cp out benchmark.PruebasEstructuras        # 16 pruebas: 16 correctas, 0 fallidas
java -Xmx3g -cp out benchmark.BenchmarkAlternativas   # genera resultados.csv
java -cp out interfaz.Interfaz                    # aplicación gráfica
```